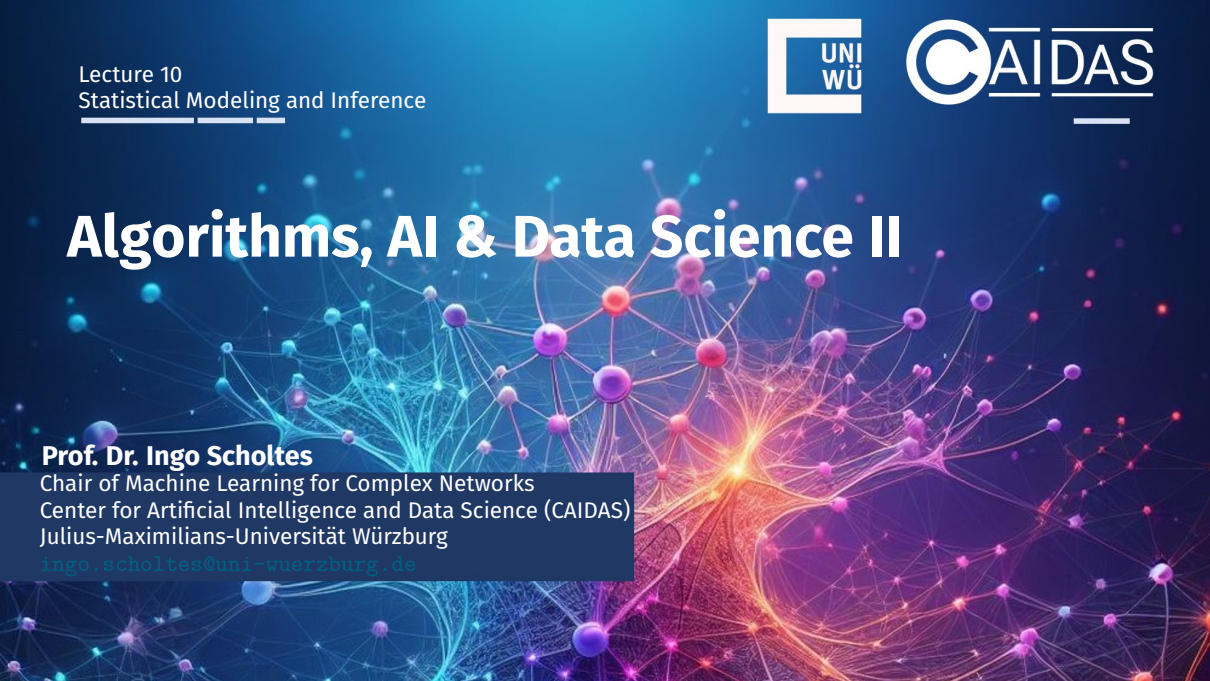
Lecture 10
Statistical Modeling and Inference

# Algorithms, AI & Data Science II



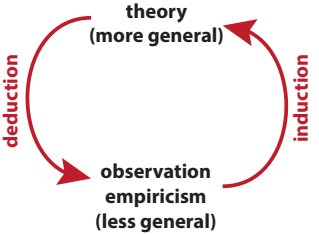
**Prof. Dr. Ingo Scholtes**
Chair of Machine Learning for Complex Networks
Center for Artificial Intelligence and Data Science (CAIDAS)
Julius-Maximilians-Universität Würzburg
ingo.scholtes@uni-wuerzburg.de

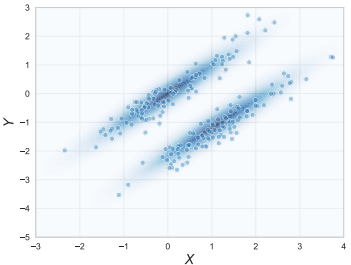
## Notes:

- **Lecture L10: Statistical Modelling and Inference** 11.06.2025
- **Educational objective:** We introduce basic discrete and continuous distributions, motivate statistical modelling and exemplify the frequentist approach to statistical inference in a simple coin tossing example.
  - Probability Distributions and Limit Theorems
  - Statistical modelling
  - Frequentist model fitting
- **Exercise sheet 05:** Frequentist statistical inference due 16.06.2025
- **Handout version:** *2025/06/03 14:45:42*

# Motivation

- we introduced probabilistic concepts to **quantitatively model uncertainty**
- basis for **statistical modelling and statistical inference**
- statistical inference allows us to **draw conclusions** based on (uncertain) data
- example for **inductive reasoning**, i.e. we use observations to derive a general “principle” or “pattern”
- foundation for **statistical learning, pattern recognition and probabilistic AI**
- **statistical hypothesis testing** enables us to formulate and assess **probabilistic propositions**




cluster detection based on inference of bivariate Gaussian distributions using Gaussian Mixture Model (GMM)

**Notes:**

# Bernoulli trials

- **Bernoulli trial** is a random experiment with binary outcome, which we denote as success (1) or failure (0)
- $p$ is probability that Bernoulli trial succeeds

**Bernoulli distribution**

- discrete distribution on $\Omega = \{0, 1\}$

$$\text{Bernoulli}(1; p) = p = 1 - \text{Bernoulli}(0; p)$$

- mean: $p$
- variance: $p(1 - p)$

**example**

"Let $X$ be outcome of a coin toss, where $X = 1$ represents heads and $X = 0$ represents tails."

$$X \sim \text{Bernoulli}(1; p)$$

where $p$ is the probability that the coin shows heads.

Jacob Bernoulli
1655 – 1705

image credit: painting by Niklaus Bernoulli, public domain

**Notes:**

# Uniform distribution

- consider random experiment with $n$ outcomes that are **equiprobable**

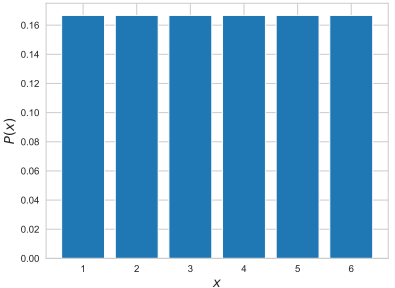
**Uniform distribution**

- discrete distribution on $\Omega = \{1, \ldots, n\}$

$$\text{Unif}(k; n) = \frac{1}{n}$$

- mean: $\frac{n+1}{2}$
- variance: $\frac{n^2-1}{12}$

- we can generalize uniform distribution to **continuous case** (e.g. on $\Omega = [0, 1]$)




uniform distribution on $[1, \ldots, 6]$

**example**

Let $X$ be random variable that maps face of fair dice to $\{1, 2, 3, 4, 5, 6\}$. Then $P(X = k) = \text{Unif}(k; 6)$, i.e.

$$X \sim Unif(k; 6)$$

$E(X) = \frac{n+1}{2} = \frac{7}{2}$ and $Var(X) = \frac{n^2-1}{12} = \frac{35}{12}$

**Notes:**

# Binomial distribution

- what is the probability to have **exactly $k$ successes in $n$ independent Bernoulli trials** with success probability $p$

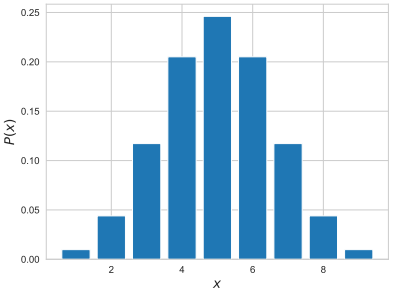
**Binomial distribution**

- discrete distribution on $\Omega = \mathbb{N}$

$$\text{Binom}(k; p, n) = \binom{n}{k} p^k (1-p)^{n-k}$$

- mean: $np$
- variance: $np(1-p)$

- $\binom{n}{k}$ counts number of different ways in which $k$ successes can be distributed among $n$ trials
- for $n = 1$ we recover **Bernoulli distribution** with success probability $p$




Binomial distribution for $n = 10$ and $p = \frac{1}{2}$

**Example**

"Let $X$ be the number of heads in ten tosses of a fair coin."

$$X \sim \text{Binom}(k; \frac{1}{2}, 10)$$

$E(X) = 5$ and $Var(X) = 2.5$

**Notes:**

# Poisson distribution

- consider events occurring randomly and independently at **mean rate** $\lambda$
- what is probability to **observe exactly** $k$ **events in a given interval**

**Poisson distribution**

- discrete distribution on $\Omega = \mathbb{N}$

$$\text{Poiss}(k; \lambda) = \frac{\lambda^k e^{-\lambda}}{k!}$$

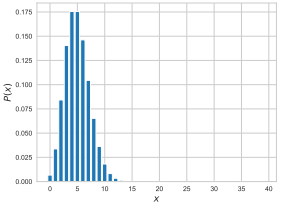
where $\lambda$ is mean rate of events in a given (time or space) interval

- mean: $\lambda$
- variance: $\lambda$

**Example**

"Let $X$ be number of asteroids larger than 15 meters that strike earth within one century, where a strike occurs on average once every 20 years."
$X \sim \text{Poiss}(k; 5)$

image credit: Fireball, Thomas Grau, public domain




Poisson distribution for $\lambda = 5$

**Notes:**

# Normal distribution

- we often observe samples of data where the **underlying distribution is unknown**
- we can calculate **mean and variance** of our sample

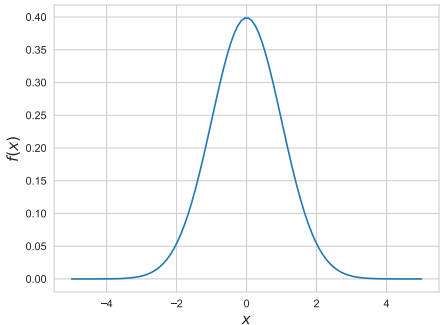
**Normal distribution**

- continuous distribution on $\mathbb{R}$

$$\text{Norm}(x; \mu, \sigma) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

- mean: $\mu$
- variance: $\sigma^2$

- used to model continuous data with **known mean and variance but unknown distribution**




Normal distribution for $\mu = 0$ and $\sigma = 1$

**Notes:**

# Central limit theorem

- consider random sample of $n$ values $X_i$ drawn from **any distribution** with known mean $\mu$ and variance $\sigma^2$
- we can calculate arithmetic mean $X := \sum_{i=1}^{n} \frac{X_i}{n}$
- central limit theorem states that $X$ is **normally distributed** around true mean $\mu$

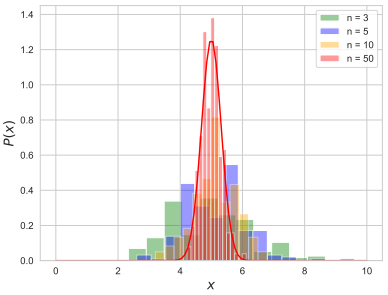
**central limit theorem** **1901**

for $n$ identically and independently distributed random variables $X_i$ with mean $\mu$ and standard deviation $\sigma$ and $X := \sum_{i=1}^{n} \frac{X_i}{n}$ we have

$$X \sim \text{Norm}(x; \mu, \sigma_X) \text{ for } n \to \infty$$

with $\sigma_X := \sqrt{\frac{\sigma}{n}}$

- for standard deviation $\sigma_X$ of sample mean $X$ we have $\sigma_X \to 0$ for $n \to \infty$




distribution of sample mean for 200 samples of $n$ realisations of $X \sim \text{Poiss}(k; 5)$ and **Normal approximation** with $\text{Norm}(x; 5, \sqrt{\frac{5}{n}})$

**Notes:**

# Approximating Binomial distributions

- Binomial distribution $B(k; p, n)$ gives probability of exactly $k$ successes in $n$ Bernoulli trials with success probability $p$
- binomial coefficient $\binom{n}{k}$ may be difficult to calculate for large $n$

**Moivre-Laplace theorem** **1738**

for $n \to \infty$ and $p = \text{const}$

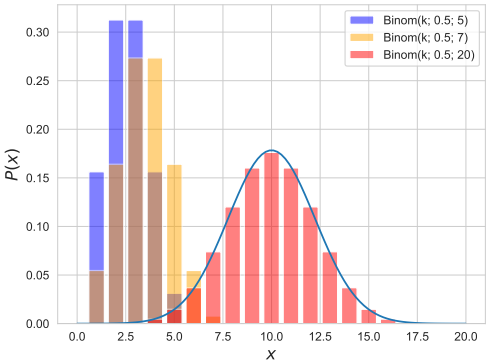
$$\text{Binom}(k; p, n) \to \text{Norm}(k; \mu, \sigma)$$

with $\mu = np$ and $\sigma = \sqrt{(np(1-p))}$

**Poisson limit theorem** **1837**

for $n \to \infty$ and $np \to \lambda = \text{const}$

$$\text{Binom}(k; p, n) \to \text{Poiss}(k; \lambda)$$

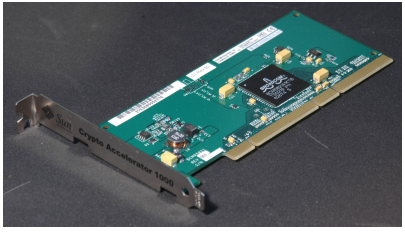



Binomial distributions for growing $n$ and $p = \frac{1}{2}$ and **Normal approximation** with $\mu = np$ and $\sigma = \sqrt{(np(1-p))}$ for $n = 20$

**Notes:**

# Randomness and algorithms?

- in principle, execution of instructions in a CPU is **deterministic**
- but we often use algorithms that make use of **randomness**
  - computational statistics
  - physics simulations
  - Bayesian inference
  - Monte Carlo algorithms
- how can we include **non-determinism** in algorithms?
- **hardware random number generators (HRNG)** use physical phenomena to generate truly random numbers
  - thermal noise in electronic circuits
  - electromagnetic noise
  - radiactive decay




Sun cryptographic accelerator card with integrated hardware number generator

image credit: Wikimedia Commons, GNU Free Documentation License

**Notes:**

# Pseudo-random generators

- can we **algorithmically** (and deterministically) generate "random numbers"?
- generated number sequence should exhibit **no patterns** and should be **indistinguishable from "true" random numbers** generated by HRNG
- we can use algorithmic (and/or arithmetic) approaches to generate **pseudo-random number sequences**
- pseudo-number generators are based on **"seed" number**, i.e. they generate same sequence when using same seed
- **middle-square method** was among first pseudo-random number generators → J von Neumann, 1949
- simple but known to be seriously flawed, i.e. **do not use for real applications**

John von Neumann, 1903 – 1957

image credit: Los Alamos National Laboratory, fair use

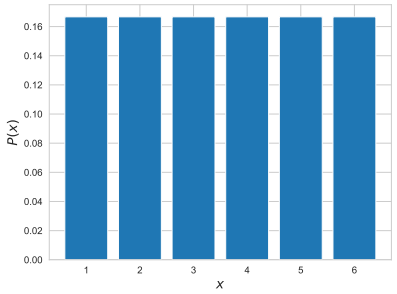
**middle-square pseudo-random generator**

**procedure** RANDOM(*seed*, *size*)
  $x \leftarrow seed$
  $seq = list()$
  **while** $x$ not in $seq$ and $len(seq) < size$ **do**
    $seq.append(x)$
    $x \leftarrow x^2$
    $x \leftarrow$ `middle_digits`$(x)$
  **end while**
  return $seq$
**end procedure**

**Notes:**

# Random uniform sampling

- we often want to **sample numbers from a continuous uniform distribution** in a given interval
- example: rolling a fair dice with $n$ faces
- basis for more complex algorithms to sample from **arbitrary continuous distributions**
- consider (pseudo-)random number generator generating sequence of numbers in range $[0, M]$
- pseudo-random sequence, i.e. we assume that values in $[0, M]$ are **equiprobable**
- to **sample from uniform distribution on** $[0, 1]$ we can divide pseudo-random numbers by $M$




**uniform sampling algorithm**

**procedure** UNIFORM(*size*, *seed* $= 0$)
  **if** *seed* $= 0$ **then**
    *seed* $\leftarrow$ *current_time*
  **end if**
  *seq* $\leftarrow$ `RANDOM(seed, size)`
  return *seq*/*M*
**end procedure**

**Notes:**

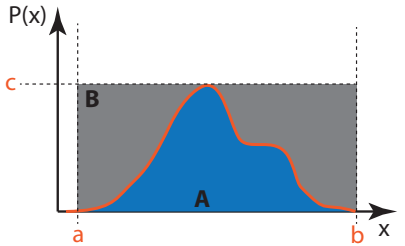
# Rejection sampling

- how can we sample from continuous distribution with **non-uniform probabilities**?
- for probability density function $P$ we define

$$a = \min\{x : P(x) > 0\}$$
$$b = \max\{x : P(x) > 0\}$$
$$c = \max_x P(X)$$

- enclose $P(x)$ by box with corners $(a, 0)$ and $(b, c)$ and consider area $A$ under $P(x)$ and $B$ above $P(x)$
- generate uniform sample $x \in [a, b]$ and $y \in [0, c]$
- **accept sample** $x$ if $(x, y) \in A$ and **reject otherwise**

→ J von Neumann, 1951



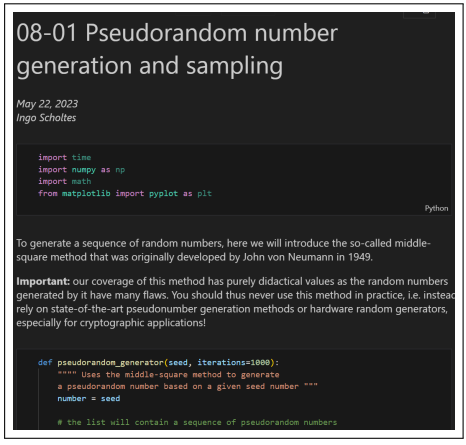

**rejection sampling algorithm**

```
procedure REJECTION_SAMPLING(P, size)
    seq = []
    while len(seq)<size do
        x, y ← Uniform(size = 2)
        x ← a + (b − a) * x
        y ← c * y
        if y ≤ P(x) then
            seq.append(x)
        end if
    end while
    return seq
end procedure
```

**Notes:**

# Practice Session

- we implement the **middle-square algorithm** to generate pseudo-random numbers
- we implement a method to **sample from a uniform distribution**
- we implement **rejection sampling** to draw samples from arbitrary distributions
- we use **scipy.stats** to generate random samples from discrete and continuous probability distributions



08-01 Pseudorandom number generation and sampling

*May 22, 2023*
*Ingo Scholtes*

```
import time
import numpy as np
import math
from matplotlib import pyplot as plt
```

Python

To generate a sequence of random numbers, here we will introduce the so-called middle-square method that was originally developed by John von Neumann in 1949.

**Important:** our coverage of this method has purely didactical values as the random numbers generated by it have many flaws. You should thus never use this method in practice, i.e. instead rely on state-of-the-art pseudonumber generation methods or hardware random generators, especially for cryptographic applications!

```
def pseudorandom_generator(seed, iterations=1000):
    """ Uses the middle-square method to generate
    a pseudorandom number based on a given seed number """
    number = seed

    # the list will contain a sequence of pseudorandom numbers
```

**practice session**

see notebooks 10-01 and 10-02 in `gitlab` repository at

→ https://gitlab.informatik.uni-wuerzburg.de/ml4nets_notebooks/2025_sose_akids2

**Notes:**

# Statistical modelling and inference

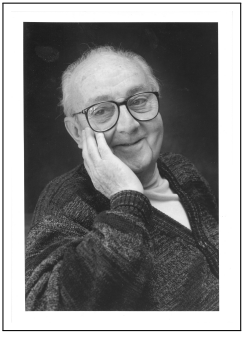
- consider sample space $\Omega$, event space $\mathcal{F} \subseteq E$ and random variable $X : \Omega \to E$
- probability distribution $P_\Theta$ with scalar or vector-valued **model parameter** $\Theta$ is called **statistical model** for $X$
- statistical model captures a possible **process that could have generated the data**
- **statistical inference**: given observations of $X$ infer the **most plausible statistical model** $P_\Theta$

**model likelihood**

for a given sample space $\Omega$ and outcome $S$ we call

$$\mathcal{L}(\Theta|S) := P_\Theta(X = S)$$

the likelihood of model $X \sim P_\Theta$ with parameter $\Theta$



George Box
1919 – 2013

"All models are wrong, but some are useful."
→ George Box

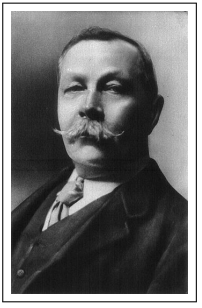
**Notes:**

# A maximally simple example

- consider **data** $x$ that counts the number of heads in $n$ tosses of a coin
- based on our observation of $x$, what can we say about the coin?
- consider **statistical model** for coin that shows heads with (unknown) probability $\Theta := p$ in each Bernoulli trial
- we can calculate the likelihood of this model based on the **binomial distribution** as

$$\mathcal{L}(p|x) = P(X = x) = \binom{n}{x} p^x (1-p)^{n-x}$$

- for which value of $p$ is the model **most plausible**?



Sir Arthur Conan Doyle
1859 – 1930

image credit: Wikimedia Commons, public domain

**likelihood maximization**

"We balance the probabilities and choose the most likely. It is the scientific use of the imagination."
→ Sherlock Holmes

**Notes:**

# Maximum Likelihood Estimation (MLE)

- for given model $P_\Theta$ and observation $x$ we can **estimate model parameter** $\hat{\Theta}$ by maximising likelihood, i.e.

$$\hat{\Theta} := \underset{\Theta}{\operatorname{argmax}}\, \mathcal{L}(\Theta|x)$$

- we **fit model parameter** $\Theta$ to the observed data
- $\hat{\Theta}$ is called **maximum likelihood estimate** of $\Theta$
- for $\mathcal{L}(\hat{\Theta}|x) = 1$ model perfectly explains our data, i.e. we observed the only possible outcome
- we can use analytical and heuristic methods to **calculate parameter $\hat{\Theta}$ that maximizes model likelihood**
- machine learning = statistics + optimization

Carl Friedrich Gauss
1777 – 1855

**Notes:**

# Group Exercise 10-01

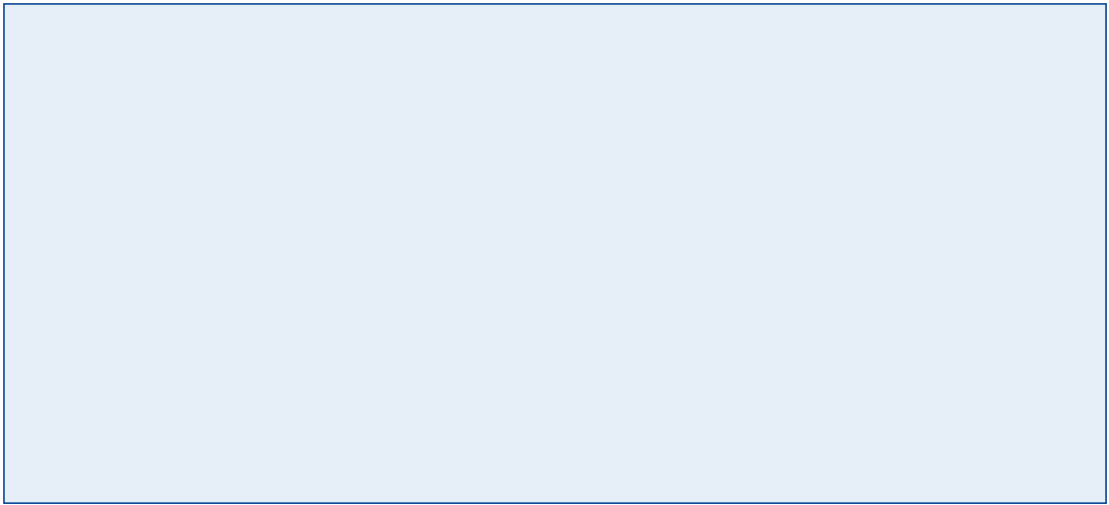


- Consider the coin tossing example from slide 3 and assume that we observe $x = 57$ heads in $n = 100$ coin tosses. Use maximum likelihood estimation to infer model parameter $p$.

**Notes:**

# Group Exercise 10-01



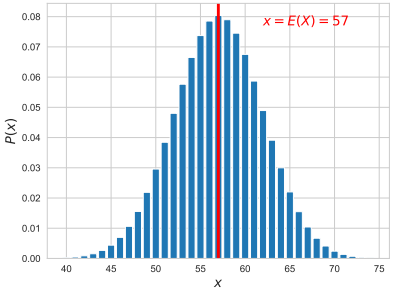


**Notes:**

- Note that the Binomial distribution has only a single extremum, which is also the global maximum. We thus know that we have found the parameter for which the likelihood is maximal.

# Frequentist model fitting

- likelihood maximisation yields **point estimate** of “most plausible” parameter of a statistical model
- in our example, this is equivalent to choosing $\hat{p}$ such that $E(X) = np = 57$
- in our example, this naturally maps to a **frequentistic interpretation** of probability
  - probability = relative frequency of event
  - 57 heads in 100 tosses $\rightarrow p = \frac{57}{100}$




$\text{Binom}(k; 100, 0.57)$ with $E(X) = 57$

**open questions**

- how confident are we in our estimate $\hat{p}$?
- what if $n$ is small, e.g. $n = 1$?

**Notes:**

# In summary

- we covered basic **discrete and continuous probability distributions**
- we considered **three limit theorems** that can be used to approximate distributions for sufficiently large numbers of samples
- statistical models allow us to use data to **reason under uncertainty**
- **statistical inference** problem refers to the process of finding “plausible” models given an observation
- we can use **maximum likelihood estimation** to find “most plausible” value of model parameters

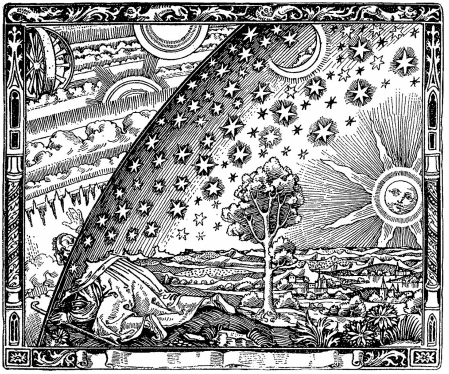
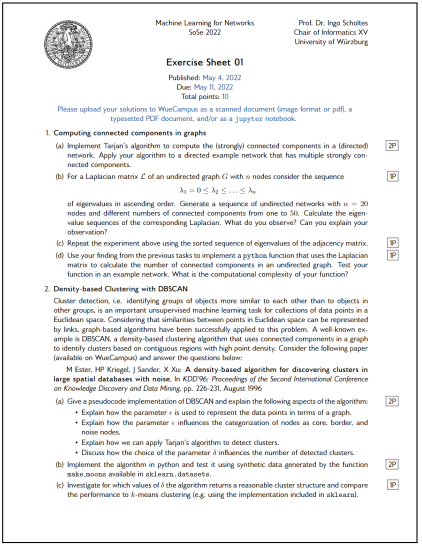


image credit: Camille Flammarion, L'Atmosphère: Météorologie Populaire, 1888, public domain

**Notes:**

# Exercise sheet 05

- **fifth exercise sheet** will be released today
- solutions are due **June 16th** (via WueCampus)
- present your solution to earn bonus points



Machine Learning for Networks
SoSe 2022

Prof. Dr. Ingo Scholtes
Chair of Informatics XV
University of Würzburg

**Exercise Sheet 01**

Published: May 4, 2022
Due: May 11, 2022
Total points: 10

Please upload your solutions to WueCampus as a scanned document (image format or pdf), a typesetted PDF document, and/or as a `jupyter notebook`.

1. **Computing connected components in graphs**

   (a) Implement Tarjan's algorithm to compute the (strongly) connected components in a (directed) network. Apply your algorithm to a directed example network that has multiple strongly connected components. [2P]

   (b) For a Laplacian matrix $\mathcal{L}$ of an undirected graph $G$ with $n$ nodes consider the sequence [1P]

   $$\lambda_1 = 0 \leq \lambda_2 \leq ... \leq \lambda_n$$

   of eigenvalues in ascending order. Generate a sequence of undirected networks with $n = 20$ nodes and different numbers of connected components from one to 5). Calculate the eigenvalue sequences of the corresponding Laplacian. What do you observe? Can you explain your observation?

   (c) Repeat the experiment above using the sorted sequence of eigenvalues of the adjacency matrix. [1P]

   (d) Use your finding from the previous tasks to implement a `python` function that uses the Laplacian matrix to calculate the number of connected components in an undirected graph. Test your function in an example network. What is the computational complexity of your function? [1P]

2. **Density-based Clustering with DBSCAN**

   Cluster detection, i.e. identifying groups of objects more similar to each other than to objects in other groups, is an important unsupervised machine learning task for collections of data points in a Euclidean space. Considering that similarities between points in Euclidean space can be represented by links, graph-based algorithms have been successfully applied to this problem. A well-known example is DBSCAN, a density-based clustering algorithm that uses connected components in a graph to identify clusters based on contiguous regions with high point density. Consider the following paper (available on WueCampus) and answer the questions below:

   M Ester, HP Kriegel, J Sander, X Xu: **A density-based algorithm for discovering clusters in large spatial databases with noise.** In *KDD'96: Proceedings of the Second International Conference on Knowledge Discovery and Data Mining*, pp. 226-231, August 1996

   (a) Give a pseudocode implementation of DBSCAN and explain the following aspects of the algorithm: [2P]
   - Explain how the parameter $\epsilon$ is used to represent the data points in terms of a graph.
   - Explain how the parameter $\epsilon$ influences the categorization of nodes as core, border, and noise nodes.
   - Explain how we can apply Tarjan's algorithm to detect clusters.
   - Discuss how the choice of the parameter $\delta$ influences the number of detected clusters.

   (b) Implement the algorithm in python and test it using synthetic data generated by the function `make_moons` available in `sklearn.datasets`. [2P]

   (c) Investigate for which values of $\delta$ the algorithm returns a reasonable cluster structure and compare the performance to $k$-means clustering (e.g. using the implementation included in `sklearn`). [1P]

**Notes:**

# Self-study questions

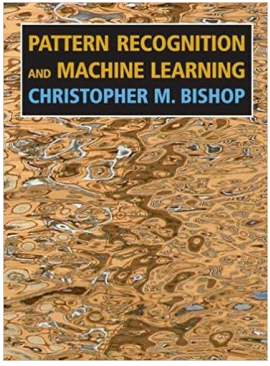
1. Name three continuous and three discrete probability distributions and give examples where their arise naturally.
2. What is the mean and variance of the Poisson distribution?
3. How can we approximate the Binomial distribution for $n \to \infty$?
4. Consider 100 independent realisations of Poisson-distributed random variable $X$ with mean and variance $\mu$. What can you see about the distribution of the sample mean?
5. Explain how we can algorithmically and deterministically generate sequences of numbers that appear to be random.
6. Explain how we can use a method that generates samples from a uniform distribution to generate samples from an arbitrary distribution.
7. Discuss potential issues of the rejection sampling algorithm for probability distributions that assign close to zero values over a large range of possible values.
8. What is a statistical model and what is the inference problem?
9. How is the likelihood function of a model defined?
10. Explain how we can find a maximum likelihood estimate of a continuous model parameter based on an analytical expression of the probability mass function.

**Notes:**

# References

**reading list**

- Hasties: **Introduction to Statistical Learning**, Springer, 2006 → Chapters 1–3
- Kevin P. Murphy: **Machine Learning - A Probabilistic Perspective**
- Christopher M. Bishop: **Pattern Recognition and Machine Learning**, Springer, 2006, → Chapters 1–3
- J von Neumann: **Various techniques used in connection with random digits. Monte Carlo methods**, Nat. Bureau Standards, Vol. 12, 1951
- N Metropolis: **The beginning of the Monte Carlo method**, Los Alamos Science, 1987
- M Mitzenmacher, E Upfal: **Probability and Computing: Randomized Algorithms and Probabilistic Analysis**, 2005
- DE Knuth: **The Art of Computer Programming**, Volume 2: Seminumerical Algorithms, Addison-Wesley, 1997



**Notes:**